Module 2) Python – Collections, functions and Modules

1) Understanding how to create and access elements in a list.

Ans:- A **list** (in most programming languages like Python) is a collection of items stored in a single variable.

Creating a list

You use square brackets []:

```
my_list = [10, 20, 30, 40]
```

Accessing elements

Each item has an index (position), starting from **0**:

```
print(my_list[0])  # Output: 10
print(my_list[2])  # Output: 30
```

2) Indexing in lists (positive and negative indexing).

Ans:- Lists use **index numbers** to access elements.

Positive indexing

- Starts from **0** (left to right)

```
my_list = [10, 20, 30, 40]

print(my_list[0])  # 10
print(my_list[2])  # 30
```

Negative indexing

- Starts from **-1** (right to left)

```
print(my_list[-1])  # 40
print(my_list[-2])  # 30
```

3) Slicing a list: accessing a range of elements.

Ans:- Slicing lets you get **multiple elements at once** using :

Syntax

```
list[start:end]
```

- start = index to begin (included)
- end = index to stop (not included)

Example

```
my_list = [10, 20, 30, 40, 50]

print(my_list[1:4])  # [20, 30, 40]
```

4) Common list operations: concatenation, repetition, membership.

Ans:- **1) Concatenation (joining lists)**

Use + to combine lists:

```
list1 = [1, 2]
list2 = [3, 4]

print(list1 + list2)  # [1, 2, 3, 4]
```

2) Repetition (repeating elements)

Use * to repeat a list:

```
my_list = [1, 2]

print(my_list * 3)  # [1, 2, 1, 2, 1, 2]
```

3) Membership (checking if item exists)

Use in or not in:

```
my_list = [10, 20, 30]

print(20 in my_list)      # True
print(40 not in my_list)  # True
```

5) Understanding list methods like append(), insert(), remove(), pop().

Ans:- **1) append()**

Adds an item to the **end** of the list:

```
my_list = [1, 2, 3]
my_list.append(4)

print(my_list)  # [1, 2, 3, 4]
```

2) insert()

Adds an item at a **specific position**:

```
my_list.insert(1, 10)

print(my_list)  # [1, 10, 2, 3, 4]
```

3) remove()

Removes a **specific value**:

```
my_list.remove(10)

print(my_list)  # [1, 2, 3, 4]
```

4) pop()

Removes and returns an item (by index, default = last):

```
my_list.pop()      # removes last element
my_list.pop(1)     # removes element at index 1

print(my_list)  # updated list
```

6) Iterating over a list using loops.

Ans:- **1) Using a for loop**

Simplest way to go through each element:

```
my_list = [10, 20, 30]

for item in my_list:
    print(item)
```

2) Using for loop with index

```
for i in range(len(my_list)):
    print(my_list[i])
```

3) Using a while loop

```
i = 0
while i < len(my_list):
    print(my_list[i])
    i += 1
```

7) Sorting and reversing a list using sort(), sorted(), and reverse().

Ans:- **1) sort() (changes original list)**

Sorts the list in ascending order:

```
my_list = [40, 10, 30, 20]
my_list.sort()

print(my_list)  # [10, 20, 30, 40]
```

2) sorted() (creates a new list)

Does not change the original list:

```
my_list = [40, 10, 30, 20]

new_list = sorted(my_list)
print(new_list)  # [10, 20, 30, 40]
```

3) reverse() (reverses order)

```
my_list.reverse()

print(my_list)  # reversed list
```

8) Basic list manipulations: addition, deletion, updating, and slicing.

Ans:- **1) Addition (adding elements)**

```
my_list = [1, 2, 3]

my_list.append(4)       # add at end
my_list.insert(1, 10)    # add at index 1

print(my_list)  # [1, 10, 2, 3, 4]
```

2) Deletion (removing elements)

```
my_list.remove(10)   # remove by value
my_list.pop()        # remove last element
del my_list[1]       # delete by index

print(my_list)
```

3) Updating (changing elements)

```
my_list[0] = 100

print(my_list)  # updated list
```

4) Slicing (getting part of list)

```
my_list = [10, 20, 30, 40, 50]

print(my_list[1:4])  # [20, 30, 40]
```

9) Introduction to tuples, immutability.

Ans:- A **tuple** is similar to a list, but it is **immutable** (cannot be changed).

Creating a tuple

Use parentheses ():

```
my_tuple = (10, 20, 30)
```

Accessing elements

Same as lists (using index):

```
print(my_tuple[0])  # 10
```

10) Creating and accessing elements in a tuple.

Ans:- **Creating a tuple**

Use parentheses ():

```
my_tuple = (10, 20, 30)
```

You can also create a tuple without parentheses:

```
my_tuple = 10, 20, 30
```

Accessing elements

Use index (same as lists):

```
print(my_tuple[0])  # 10
print(my_tuple[2])  # 30
```

11)Basic operations with tuples:concatenation, repetition, membership.

Ans:- **1) Concatenation (joining tuples)**

Use + to combine tuples:

```
t1 = (1, 2)
t2 = (3, 4)

print(t1 + t2)  # (1, 2, 3, 4)
```

2) Repetition (repeating tuple)

Use * to repeat elements:

```
t = (1, 2)

print(t * 3)  # (1, 2, 1, 2, 1, 2)
```

3) Membership (checking element)

Use in or not in:

```
t = (10, 20, 30)

print(20 in t)      # True
print(40 not in t)  # True
```

12)Accessing tuple elements using positive and negative indexing.

Ans:- Tuples use the same indexing rules as lists.

1) Positive indexing

Starts from **0 (left to right)**:

```
my_tuple = (10, 20, 30, 40)

print(my_tuple[0])  # 10
print(my_tuple[2])  # 30
```

2) Negative indexing

Starts from **-1 (right to left)**:

```
print(my_tuple[-1])  # 40
print(my_tuple[-3])  # 20
```

13)Slicing a tuple to access ranges of elements.

Ans:- Slicing lets you access **multiple elements** from a tuple using :

Syntax

tuple[start:end]

- start → included
- end → not included

Example

```
my_tuple = (10, 20, 30, 40, 50)

print(my_tuple[1:4])  # (20, 30, 40)
```

14) Introduction to dictionaries: key-value pairs.

Ans:- A **dictionary** is a collection in Python that stores data in **key-value pairs**.

Creating a dictionary

Use curly braces {}:

```
my_dict = {
   "name": "Asha",
   "age": 20,
   "city": "Surat"
}
```

Key–Value concept

- **Key** → name or label (unique)
- **Value** → data stored for that key

Example:

```
print(my_dict["name"])  # Asha
print(my_dict["age"])   # 20
```

15) Accessing, adding, updating, and deleting dictionary elements.

Ans:- A dictionary stores data in **key-value pairs**.

1) Accessing elements

Use the key:

```
my_dict = {"name": "Asha", "age": 20}

print(my_dict["name"])  # Asha
```

2) Adding elements

Add a new key-value pair:

```
my_dict["city"] = "Surat"

print(my_dict)
```

3) Updating elements

Change value using key:

```
my_dict["age"] = 21

print(my_dict)
```

4) Deleting elements

```
del my_dict["city"]   # delete specific key
```

16) Dictionary methods like keys(), values(), and items().

Ans:- Dictionaries have useful methods to get keys, values, or both.

1) keys()

Returns all **keys** in the dictionary:

```
my_dict = {"name": "Asha", "age": 20}

print(my_dict.keys())
# dict_keys(['name', 'age'])
```

2) values()

Returns all **values**:

```
print(my_dict.values())
# dict_values(['Asha', 20])
```

3) items()

Returns **key-value pairs** as tuples:

```
print(my_dict.items())
# dict_items([('name', 'Asha'), ('age', 20)])
```

17)Iterating over a dictionary using loops.

Ans:- you can loop through a dictionary to access keys, values, or both.

1) Loop through keys (default)

```
my_dict = {"name": "Asha", "age": 20}

for key in my_dict:
    print(key)
```

2) Loop through values

```
for value in my_dict.values():
    print(value)
```

3) Loop through key-value pairs

```
for key, value in my_dict.items():
    print(key, value)
```

18) Merging two lists into a dictionary using loops or zip().

Ans:- You can combine two lists (keys + values) into a dictionary using **loops** or zip().

1) Using zip() (easy method)

```
keys = ["name", "age", "city"]
values = ["Asha", 20, "Surat"]

my_dict = dict(zip(keys, values))

print(my_dict)
# {'name': 'Asha', 'age': 20, 'city': 'Surat'}
```

2) Using a loop

```
keys = ["name", "age", "city"]
values = ["Asha", 20, "Surat"]

my_dict = {}

for i in range(len(keys)):
    my_dict[keys[i]] = values[i]

print(my_dict)
```

19) Counting occurrences of characters in a string using dictionaries.

Ans:- you can use a dictionary to count how many times each character appears in a string.

Example using a loop

```
text = "apple"
count = {}

for char in text:
```

```
    if char in count:
        count[char] += 1
    else:
        count[char] = 1

print(count)
# {'a': 1, 'p': 2, 'l': 1, 'e': 1}
```

20) Defining functions in Python.

Ans:- A **function** is a block of code that performs a specific task and can be reused.

Syntax

```
def function_name():
    # code block
    return
```

Example

```
def greet():
    print("Hello!")

greet()
```

Function with parameters

```
def add(a, b):
    return a + b

print(add(5, 3))  # 8
```

21)Different types of functions: with/without parameters, with/without return values.

Ans:- Functions can be classified based on **parameters** and **return values**.

1) Without parameters, without return value

```
def greet():
    print("Hello!")

greet()
```

- No input
- No output (only prints)

2) With parameters, without return value

```
def greet(name):
    print("Hello", name)

greet("Asha")
```

- Takes input
- Only prints, does not return

3) Without parameters, with return value

```
def get_number():
    return 10

print(get_number())
```

- No input
- Returns a value

4) With parameters, with return value

```
def add(a, b):
    return a + b
```

```
print(add(5, 3))
```

- Takes input
- Returns result

22) Anonymous functions (lambda functions).

Ans:- A **lambda function** is a small, anonymous function (no name) used for simple operations.

Syntax

```
lambda arguments: expression
```

Example 1: Simple addition

```
add = lambda a, b: a + b

print(add(5, 3))  # 8
```

Example 2: Square of a number

```
square = lambda x: x * x

print(square(4))  # 16
```

Example 3: Using with map()

```
nums = [1, 2, 3, 4]

squares = list(map(lambda x: x*x, nums))

print(squares)
```

23) Introduction to Python modules and importing modules.

Ans:- A **module** in Python is simply a file that contains Python code (functions, variables, or classes). It helps you organize code into reusable parts.

Why use modules?

- Reuse code easily
- Keep programs organized
- Save time and effort

Types of modules:

- **Built-in modules** (already available in Python)
  Example: math, random, datetime
- **User-defined modules** (created by you)

How to import modules:

1. Import the whole module

```
import math
print(math.sqrt(16))
```

2. Import specific function

```
from math import sqrt
print(sqrt(16))
```

3. Import with alias (short name)

```
import math as m
print(m.sqrt(16))
```

24) Standard library modules: math, random.

Ans:- Python has many **built-in (standard library) modules** that you can use without installing anything. Two important ones are:

1. math module

The math module provides mathematical functions.

Common functions:

- math.sqrt(x) → square root

- math.pow(x, y) → power (x^y)
- math.ceil(x) → rounds up
- math.floor(x) → rounds down

Example:

```
import math

print(math.sqrt(25))   # 5.0
print(math.pow(2, 3))  # 8.0
print(math.ceil(4.3))  # 5
print(math.floor(4.9)) # 4
```

2. random module

The random module is used to generate random numbers.

Common functions:

- random.randint(a, b) → random integer between a and b
- random.random() → random float between 0 and 1
- random.choice(list) → picks a random item

Example:

```
import random

print(random.randint(1, 10))   # e.g. 7
print(random.random())        # e.g. 0.5432

items = ["apple", "banana", "mango"]
print(random.choice(items))    # e.g. mango
```

25) Creating custom modules.

Ans:- A **custom module** is simply your own Python file that contains reusable code like functions, variables, or classes.

Step 1: Create a Python file (module)

Create a file named mymodule.py

```
# mymodule.py

def greet(name):
    return "Hello " + name

def add(a, b):
    return a + b
```

Step 2: Import the module in another file

```
import mymodule

print(mymodule.greet("Asha"))
print(mymodule.add(5, 3))
```

Output:

```
Hello Asha
8
```

Alternative: Import specific functions

```
from mymodule import greet

print(greet("Rahul"))
```